TD 16 : Algorithmes d'approximation

Exercice 1 (Couplage de poids maximal)
On considère le problème `Couplage de Poids Maximal` (`CPM`) suivant :

Instance :	Un graphe non orienté $G=(S,A)$ muni d'une fonction p de poids à valeurs dans $\mathbb{R}^+$
Solution :	Un couplage C
Fonction à maximiser :	$\sum_{a\in C} p(a)$

On propose l'algorithme suivant :

Algorithme 1 Glouton(S,A,p)

```
C ← ∅
pour a ∈ A par ordre décroissant de poids faire
    si C ∪ {a} est un couplage alors
        C ← C ∪ {a}
    fin si
fin pour
return C
```

Question 1 : Quelle est la complexité de cet algorithme ?

Soit C_{OPT} une solution optimale au problème `CPM` et C la solution renvoyée par l'algorithme.
Pour chaque arête $a^\star$ de C_{OPT}, on définit $g(a^\star)$ de la façon suivante :

$$g(a^\star)\in C \quad g(a^\star)\cap a^\star \neq \emptyset \quad f(g(a^\star))\geq f(a^\star)$$

Question 2 : Justifier que cette fonction g est bien définie.
Question 3 : Montrer qu'au plus deux éléments de C_{OPT} ont même image par g.
Question 4 : En déduire que l'algorithme proposé est une 2-approximation du problème `CPM`.
Question 5 : Proposer un exemple de graphe pour lequel cette borne est atteinte.

Exercice 2 (Sous-Somme)
On considère le problème `Sous-Somme` suivant :

Instance :	Une liste L de n entiers $a_1, a_2, \dots a_n$
	Un entier cible C.
Solution :	un sous-ensemble $I\subseteq [\![1,n]\!]$ tel que $\sum_{i\in I} t_i \leq C$
Fonction à maximiser :	$\sum_{i\in I} t_i$

On propose l'algorithme suivant :

Algorithme 2 Algo1(L,C)

```
Trier L.
S ← 0.
pour i = n à i = 1 faire
    si a_i ≤ C − S alors
        S ← S + a_i.
    fin si
fin pour
return S
```

Question 1 : Déterminer la complexité temporelle de l'algorithme précédent.
Question 2 : Montrer que cet algorithme est une $\frac{1}{2}$-approximation de `SOMME PARTIELLE`.
Pour $\varepsilon>0$, on considère l'algorithme suivant :

Algorithme 3 Algo1(L, C, ε)

```
S ← {0}.
pour i = 1 à i = n faire
    S' ← ∅.
    pour x ∈ S faire
        S' ← S' ∪ {x, x + a_i}.
    fin pour
    x ← min S'.
    S ← {x}.
    pour y ∈ S' par ordre croissant faire
        si x + εC/n < y ≤ C alors
            x ← y.
            S ← S ∪ {y}.
        fin si
    fin pour
fin pour
return max S
```

Question 3 : Montrer que la propriété « $|S| \leq \frac{n}{\varepsilon}$ et tout élément de S est inférieur ou égal à C » est un invariant de la première boucle.
Question 4 : En déduire que l'algorithme s'exécute en temps polynomial en $\frac{n}{\varepsilon}$.
Question 5 : Que peut-on dire de cet algorithme ?